

Rise of the machines

Understanding the basics of Tensorflow

Arunava Roy Choudhury |
feedback@digit.in

T

he year is ____, your home assistant wakes you up, plays your favorite tune as you freshen up and

compiles the to-do list for the day. Your favorite breakfast is ready for you by the time you are ready, tailored to your calorie intake and just as you are done, your health coach is notified of the meal timing. Preparing to leave for work, you unplug your electric car, command it to drive out of the garage as you hurriedly imprint lock the door. As you zip down the subterranean highway idly browsing the daily news updates of your preference you are alerted to the predicted arrival to your office and an automatic update is sent to your secretary. Welcome to the future, technology has evolved and developed intelligence, complimenting and augmenting civilization. Except this isn't the future, the year is 2018 and all of the technology is already here, driven and orchestrated by what is being called the biggest leap in recent history, artificial intelligence

TENSORFLOW

The process of designing artificial intelligence is mind bogglingly complex in its entirety, that does not mean that we as aspiring developers cannot approach it at all.

In 2010, Google launched its now extremely popular machine-learning

framework Tensorflow, that simplifies the technique of machine learning and neural networks, which is a branch of artificial intelligence that uses mathematical structures that mimic (to some extent) our human brains to solve certain types of problems.

We will attempt to design a NN (neural network) that given an image tries to classify it as a circle, square, star or a triangle. Let us approach this problem step by step.

NETWORK ARCHITECTURE

When designing the architecture of a neural network we have to decide on a number of things: How do you arrange layers? which layers to use? how many neurons to use in each layer etc.? Designing the architecture is a complicated topic and takes a lot of research, however there are a number of standard architectures which work well for many problems. Examples being AlexNet, GoogleNet, VGG etc.

Once you have decided the architecture, the second task is to identify the weights(w) and biases(b) or the parameters of the network. The objective of the training is to get the best possible values of the all these parameters. For example, while trying to build a classifier between dog and cat, we are aiming to find parameters such that output layer gives out probability of dog higher than cat for all images of dogs and probability of cat higher than dog for all images of cats. One of the ways to find the best set of parameters is a process called Backward propagation. Here, you start with a random set of parameters and keep changing these weights such that for every training image we get the maximum correct output. There are many optimiser

methods to change the weights and gradient descent is one such method.

We basically want to maximise the total correct classifications by the network i.e. we want to make changes such that the number of correct classifications by the network increases. So we create a single variable called cost which informs us if the training is helping. Typically cost is defined in such a way that as the cost is reduced, the accuracy increases. So, we keep track of cost and do many iterations till cost stops decreasing. One of the simplest ways to define cost is mean root square cost. It is defined as:

$$\text{cost} = 0.5 \sum_{i=0}^n (y^{\text{actual}} - y^{\text{prediction}})^2$$

After training is complete, these parameters and architecture will be saved in a binary file. Later, when we get a new image of shape to classify, we load the model in the same network and calculate the probability of the new image. This is called inference.

Finally, the pass of a single complete set of training data is called one epoch, i.e. in one epoch the network sees all the training images once.

BUILDING A SMALL NEURAL NETWORK BASED IMAGE CLASSIFIER

The Neural Network that we will implement is very small and intended only for learning purposes so that you can train this on a CPU as well. While training, images from all the classes are fed to a convolutional layer which is followed by 2 more convolutional layers. After convolutional layers, we flatten the output and add two fully connected layers in the end. The second fully connected layer has only four outputs which represent the probability of an image being a circle, a square, a triangle or a star.

Pre-requisites

- i) OpenCV: We use openCV to read images so you will have to install it.
- ii) Softmax: We shall apply the Softmax function to the output of our convolutional neural network in order to convert the output to the probability for each class.